09 · Capstone-Projekt & Architekturmuster

Architektur, Datenvertrag und begründete Abnahme · EZS · DHBW Stuttgart

Prof. Dr.-Ing. Daniel Klünder

Folie 1 / 50

09 · Aus Bausteinen wird ein System

- Capstone-Projekt und Architekturmuster
- Ereignisse erfassen, Stände austauschen, Ergebnisse prüfen
- Rollen, Datenvertrag und Zeitvertrag gemeinsam entwerfen
- Ein endlicher Demonstrator auf dem NUCLEO-H755ZI-Q

Notizen 1 / 50

Capstone bezeichnet das Abschlussprojekt, in dem die bisher einzeln behandelten Bausteine zusammenwirken. Eine Architektur beschreibt Aufgaben, ihre Verteilung und die Verträge zwischen ihnen. Auf dem NUCLEO-H755ZI-Q verwenden wir weiterhin den STM32H755 mit zwei Kernen: Cortex-M4 mit FreeRTOS, dem bekannten Echtzeitbetriebssystem, und Cortex-M7 ohne Kernel, also Bare Metal. Ein synthetischer Sensor liefert Zahlen; eine reale physische Regelstrecke wird nicht vorausgesetzt.
Heute soll aus einem plausiblen Blockbild ein prüfbarer Entwurf werden. Dazu gehören die Bedeutung jeder Nachricht, zulässige Auslassungen, Fristen und die Aussagekraft einer Beobachtung. Eine korrekt gezählte Ereignisfolge beantwortet noch keine Zeitfrage. Ein schneller Testlauf beweist umgekehrt keine Vollständigkeit der Daten. Wir behandeln diese Fragen getrennt und führen sie am Ende in einer Abnahmetabelle zusammen. Der Demonstrator ist endlich, damit seine Bilanz nach Abschluss beider Kerne ohne laufende Änderungen geprüft werden kann.

Folie 2 / 50

Die bisherigen Vorlesungen sind die Grundlage

Bisher erarbeitet	Anwendung heute
Frist, Antwortzeit und Scheduling	messbare Teilanforderungen
Task, Queue und Interrupt	Ereigniskette auf M4
Shared Memory, Cache und MPU	gemeinsamer Datenblock
Trace, HSEM und Watchdog	Diagnose, Besitz und Fortschritt

Notizen 2 / 50

Scheduling ist die Zuteilung von Prozessorzeit. Eine Task ist ein vom Kernel verwalteter Ausführungspfad, eine Queue eine Nachrichtenwarteschlange mit begrenztem Platz. Ein Interrupt unterbricht den normalen Ablauf zugunsten eines Handlers. M4 bezeichnet den Cortex-M4. Shared Memory ist gemeinsam erreichbarer Speicher; der Cache enthält lokale Kopien. MPU heißt Memory Protection Unit, Speicherschutzeinheit eines Kerns. Trace bedeutet geordnete Ereignisaufzeichnung, HSEM Hardware Semaphore, das Hardware-Schloss. Ein Watchdog überwacht das rechtzeitige Nachladen eines Zählers.
Diese Vorkenntnisse aus Vorlesungen 1–8 genügen. Neu ist die Verknüpfung: Der Empfang einer Nachricht, ihre Veröffentlichung und ihr späteres Kopieren sind verschiedene Ereignisse. Jede Stufe hat ihre eigenen möglichen Verzögerungen und Auslassungen. Der Capstone wiederholt die Schutzmechanismen deshalb nur an der Stelle, an der ihr Vertrag gebraucht wird. Die folgende Orientierung zeigt, welche Ergebnisse Sie nach diesem Termin begründen können.

Folie 3 / 50

Fünf konkrete Ergebnisse dieses Termins

1. Architekturfragen voneinander unterscheiden
2. Queue und Latest-Value passend auswählen
3. Ereignis- und Publikationsnummern getrennt bilanzieren
4. Antwortzeit, Datenalter und Haltezeit richtig messen
5. Ein Kriterium mit Bedingungen und Prüfstatus abnehmen

Notizen 3 / 50

Latest-Value bezeichnet einen Speicherplatz für den jeweils neuesten veröffentlichten Stand. Eine Queue bewahrt dagegen angenommene Nachrichten bis zu ihrer Entnahme. Die Auswahl hängt davon ab, ob jede Zwischeninformation gebraucht wird. Eine Ereignisnummer entsteht am Eingang; eine Publikationsnummer erst bei erfolgreicher Veröffentlichung. Diese beiden Nummern erklären unterschiedliche Lücken.
Antwortzeit ist hier die Zeit zwischen benanntem Eingang und gefordertem Abschluss. Datenalter misst, wie alt die kopierte Information an ihrem festgelegten Bezugspunkt ist. Haltezeit beschreibt nur die Dauer des bestätigten Schlossbesitzes. Ein Prüfstatus unterscheidet beispielsweise einen gültigen beobachteten Lauf von einem noch nicht ausgeführten Versuch. Am Ende liefern Sie daher nicht nur Code, sondern Lastbeschreibung, Protokoll, Rechenbeispiele und eine begründete Ergebnisübersicht. Der gleiche Entwurf kann logisch korrekt sein und trotzdem eine Zeitgrenze verletzen. Genau diese Unterscheidung soll der Abschlussversuch sichtbar machen.

Folie 4 / 50

Vier Schulstunden: Entwurf und Abnahme

Tätigkeit	Minuten
Architektur und Schnittstellen · Folien 1–23	45
Capstone-Vertrag und Verifikation · Folien 24–50	45
Vier Papieraufgaben mit Besprechung	50
Vorbereiteter STM32-Versuch	25
Quiz und gemeinsame Klärung	15
Gesamt	180

Notizen 4 / 50

Vier Schulstunden mit je 45 Minuten ergeben 180 Minuten. STM32 ist die verwendete Mikrocontrollerfamilie. Die 25 Labor-Minuten setzen ein vorbereitetes Dual-Core-Projekt voraus: zwei getrennte Firmwares, gemeinsamer Start und der Speichervertrag der vorherigen Versuche. Die erstmalige Installation von CubeMX oder der Entwicklungsumgebung ist darin nicht enthalten.
Der erste Block führt von Architekturentscheidungen zu Schnittstellen. Der zweite verfolgt eine einzelne Ereigniskette und macht ihre Abnahmebedingungen ausdrücklich. In den Papieraufgaben werden unter anderem Queuebelegung, Mailboxbilanz, Zeitabstände und Filterwerte ausgerechnet. Der Programmierteil prüft dieselben Begriffe an einer endlichen Implementierung. Bei fehlendem Board bleiben die vier Pflichtaufgaben vollständig lösbar. Das Quiz fragt anschließend nach Unterscheidungen, die auch in einer Klausur relevant sind. Hardwarebeobachtungen werden dabei nicht durch Modellzahlen ersetzt.

Folie 5 / 50

Ein Blockbild braucht Verträge

Übergang	Zu klären
Ereignis → Queue	Annahme, Reihenfolge, voller Puffer
Worker → Mailbox	Einheit, Besitz, vollständiger Datensatz
Mailbox → Filter	Auslassungen, Alter, Gültigkeit
Fehler → Reaktion	Nachweis, Frist, Wiederanlauf

Pfeile beschreiben Beziehungen, keine garantierten Laufzeiten.

Notizen 5 / 50

Ein Worker ist der Ausführungspfad, der angenommene Arbeit verarbeitet. Eine Mailbox ist hier der gemeinsam erreichbare Speicher für Latest-Value, den neuesten Stand. Ein Datensatz besteht aus

Notizen, Fortsetzung 5 / 50

Nutzwert und den zugehörigen Kennungen. Die Tabelle fragt bei jedem Übergang, was vorher bekannt sein muss und was danach tatsächlich zugesagt wird.
Ein Pfeil von Ereignis zur Queue bedeutet beispielsweise nicht, dass jede physische Flanke angenommen wird. Der Eingang kann zusammengefasste Hardwareereignisse sehen, und die Queue kann voll sein. Ein Pfeil zur Mailbox bedeutet nicht, dass jeder veröffentlichte Zwischenwert später kopiert wird. Ein Fehlerpfeil braucht wiederum eine definierte Reaktion und ihre maximale Zeit. HSEM, Hardware Semaphore, schützt den gemeinsamen Abschnitt; die MPU, Memory Protection Unit, prüft Adressrechte dieses Kerns. Keines davon füllt die fehlenden Verträge von selbst. Zuerst formulieren wir deshalb die Anforderungen in messbaren Teilgrößen.

Folie 6 / 50

Aus „schnell und verlustfrei“ werden Anforderungen

Lehrziele unter ausdrücklich benannter Normallast:

- 100 Eingabeereignisse je Sekunde, Queue-Tiefe 8
- Publikationsantwort höchstens 1 ms
- Alter eines neu kopierten Stands höchstens 50 ms
- HSEM-Haltezeit je Seite höchstens 50 µs
- Jede Abweisung und jede erlaubte Ersetzung wird gezählt

Das sind Anforderungen, keine bereits gemessenen Ergebnisse.

Notizen 6 / 50

100 Ereignisse pro Sekunde bedeuten einen nominalen Abstand von zehn Millisekunden. Die Queue ist der lokale Nachrichtenpuffer. HSEM bedeutet Hardware Semaphore. Die Publikationsantwort reicht vom definierten Eingangspunkt bis zur Veröffentlichung. Das Snapshotalter bezieht sich nur auf wirklich kopierte neue Stände; ein Snapshot ist eine lokale Kopie eines konsistenten Datensatzes. Die Haltezeit beginnt nach dem erfolgreichen Erwerb und endet bei der Freigabe.
Diese Zahlen sind Lehrfestlegungen. Sie gelten im Normalversuch mit vorbereitetem Eingang und endlichen Datenmengen. Der Queue-Sättigungsversuch und ein dauerhaft blockierter Partner haben andere Erwartungen. „Verlustfrei“ wäre für eine Latest-Value-Mailbox die falsche Forderung: Sie darf Zwischenstände ersetzen. Gefordert ist stattdessen eine vollständige Erklärung der gezählten Übergänge. Auch die Ausgabedauer bekommt bei Bedarf ein eigenes Ziel; sie wird nicht heimlich in die 50 Millisekunden hineingelesen. Die nächsten Folien wiederholen die Zeitbegriffe, bevor wir Architekturvarianten vergleichen.

Folie 7 / 50

Antwortzeit und eigene Rechenzeit

Ein synthetischer Job:
$r = 1$ ms, $s = 2$ ms, $f = 7$ ms, $d = 8$ ms.

$$R = f - r = 6\,\text{ms}, \qquad C = 3\,\text{ms}$$

- r: Freigabe, s: erster Start, f: Abschluss
- d: absolute Deadline, C: eigene Rechenzeit
- Frist eingehalten, Reserve $d - f = 1$ ms

Wartezeit und fremde Arbeit gehören nicht zu C.

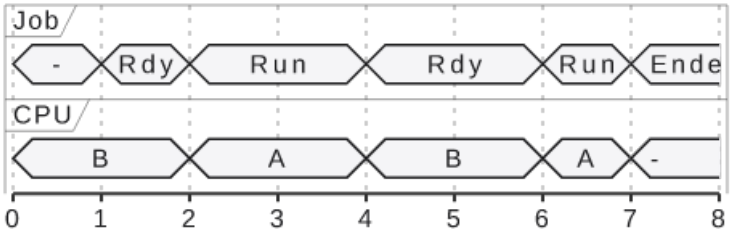
Notizen 7 / 50

Ein Job ist eine konkrete Aktivierung einer Task. r bezeichnet seine Freigabe, s den ersten Ausführungsbeginn, f den Abschluss und d die absolute Deadline, also die geforderte Abschlusszeit. C ist die eigene Prozessorarbeit des Jobs. R ist seine Antwortzeit. Die Modellzahlen stammen aus dem bekannten Zeitbeispiel: Eigene Ausführung liegt von zwei bis vier und von sechs bis sieben Millisekunden, insgesamt drei Millisekunden. Von Freigabe eins bis Abschluss sieben vergehen dagegen sechs Millisekunden.
Die Deadline acht ist eingehalten; es bleibt eine Millisekunde Reserve. Zwischen vier und sechs kann eine andere Aufgabe laufen. Diese fremde Arbeit ist Bestandteil der verstrichenen Antwortzeit, aber nicht der eigenen Rechenzeit des Jobs. Auch das anfängliche Warten von eins bis zwei gehört nur zur Antwortzeit. Ein schneller Filter mit drei Millisekunden eigener Arbeit kann daher eine zu enge Ende-zu-Ende-Frist verletzen, wenn die anderen Anteile groß sind. Das Timing-Diagramm zeigt genau diese Trennung.

Folie 8 / 50

Ready und Running bleiben verschieden

Ein-Kern-Lehrmodell in ms · Kontextwechselkosten hier null.



Notizen 8 / 50

Das Timing-Diagramm hat eine waagerechte Millisekundenachse von null bis acht. Die obere Spur beschreibt den Jobzustand. Rdy steht für Ready, ausführungsbereit, und Run für Running, tatsächlich laufend. Die untere Spur zeigt die CPU, Central Processing Unit, also den Kern: A ist der betrachtete Job

Notizen, Fortsetzung 8 / 50

und B fremde Arbeit. Die Namen bezeichnen hier keine Hardwarekerne. Der Job wird bei eins bereit, läuft von zwei bis vier, wartet ausführungsbereit bis sechs und läuft erneut bis sieben. Ende markiert seinen Abschluss; der Gedankenstrich eine hier nicht weiter modellierte Aktivität.
Die beiden Laufsegmente ergeben drei Millisekunden eigene Arbeit. Freigabe bis Ende ergibt sechs Millisekunden Antwortzeit. Die Deadline liegt bei acht; der abschließende freie Abschnitt zeigt die eine Millisekunde Reserve. Kontextwechselkosten sind ausdrücklich null im Modell. Eine reale Messung müsste sie sinnvoll zuordnen. Das Bild ist ein Schedulingmodell und keine Traceaufnahme. Es zeigt, warum der Zustand einer Task und die momentan laufende Kernarbeit gemeinsam betrachtet werden müssen.

Folie 9 / 50

Jitter nur mit benannter Größe

Lehrstichprobe von Publikationsantworten:

$$R_{\text{pub}} = 0{,}50;\ 0{,}75;\ 0{,}90\,\text{ms}$$

$$J_R = \max R_{\text{pub}} - \min R_{\text{pub}} = 0{,}40\,\text{ms}$$

- beobachtetes Maximum: 0,90 ms
- Ziel der Stichprobe: höchstens 1 ms
- kein WCET-Beweis aus drei Werten

Periodenabweichung wäre eine andere Jitterdefinition.

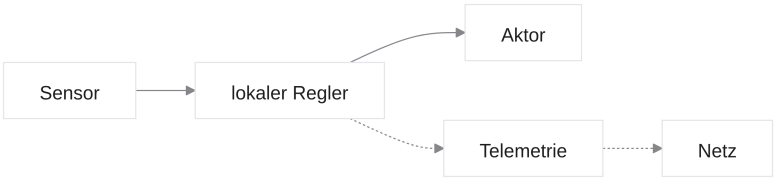
Notizen 9 / 50

Jitter bezeichnet eine Schwankung einer ausdrücklich festgelegten Zeitgröße. Hier ist J R die Spanne der beobachteten Publikationsantworten R pub. Die drei Werte sind didaktische Zahlen, keine Aufnahme auf dem Board. Der größte ist 0,90 und der kleinste 0,50 Millisekunden; die Differenz ist 0,40 Millisekunden. Alle drei Werte liegen unter dem Lehrziel von einer Millisekunde.
Das beobachtete Maximum ist jedoch keine allgemeine obere Grenze. WCET bedeutet Worst-Case Execution Time, obere Ausführungszeit unter benannten Annahmen, und betrifft zusätzlich ein festgelegtes Stück eigener Arbeit. Antwortzeit kann andere Verzögerungen enthalten. Eine Periodenabweichung würde stattdessen die Ereignisabstände mit einem Sollabstand vergleichen. Diese zwei Definitionen dürfen nicht mit derselben unbeschrifteten Zahl vermischt werden. Ein Kernel, ein schneller Prozessor oder ausgeschalteter Cache ersetzen keinen vollständigen Zeitnachweis. Die Architektur muss nun entscheiden, wo die fristkritische Arbeit ausgeführt wird.

Folie 10 / 50

CPS: lokaler Wirkungspfad und Telemetrie

Eine physische Regelstrecke ist das allgemeine Bild, kein Teil unseres Zahlengenerators.



Notizen 10 / 50

CPS bedeutet Cyber-Physical System, ein System mit gekoppelter Rechnung und physischer Wirkung. Ein Aktor setzt eine Stellgröße physisch um. Telemetrie überträgt Beobachtungen nach außen. Das Architekturdiagramm liest sich von links nach rechts: Die durchgezogenen Pfeile verbinden Sensor, lokalen Regler und Aktor. Die gestrichelten Pfeile bilden den zusätzlichen Telemetriepfad zum Netz. Sie kennzeichnen einen anderen Vertrag, keine langsameren Pfeilgeschwindigkeiten.
Der lokale Kreis kann eigene enge Fristen haben. Eine variable Netzlatenz soll diese Fristen nicht heimlich bestimmen. Fällt das Netz aus, muss die Anwendung festlegen, wie der lokale Kreis weiterarbeitet oder in einen definierten Zustand übergeht. Ein CPS braucht kein öffentliches Netz, und ein Telemetrieausfall ist nicht automatisch ein Fehler der lokalen Berechnung. Im Kursdemonstrator gibt es nur synthetische Zahlen und keine angeschlossene Maschine. Das Bild erklärt die allgemeine Architekturfrage. Die folgende Rechnung zeigt einen möglichen Grund, Daten bereits nahe an ihrer Quelle auszuwerten.

Folie 11 / 50

Edge: eine Datenrate transparent rechnen

Modell: 40 000 Proben/s · 1 000 Sensoren · 2 Byte/Probe.

$$40000 \cdot 1000 \cdot 2 = 80\,000\,000\ \text{Byte/s}$$

$$80\ \text{MB/s} = 640\ \text{Mbit/s}$$

- MB dezimal: 10^6 Byte
- Nutzdaten ohne Protokoll; keine STM32-Leistungsangabe

Notizen 11 / 50

Edge bezeichnet Verarbeitung nahe an der Datenquelle, beispielsweise auf dem Gerät oder einem örtlichen Gateway. Die Rechnung verwendet frei gesetzte Lehrannahmen. 40000 Proben je Sekunde mal 1000 Sensoren mal zwei Byte je Probe ergibt 80 Millionen Byte pro Sekunde. MB bedeutet hier dezimale Megabyte mit einer Million Byte. Ein Byte enthält acht Bit, daher sind es 640 Millionen Bit pro Sekunde oder 640 Megabit pro Sekunde. MiB wäre Mebibyte mit 1048576 Byte und eine andere Einheit.

Notizen, Fortsetzung 11 / 50

Die Zahl ist ausschließlich Nutzdaten. Adressierung, Rahmen, Prüfinformationen und Pausen könnten den benötigten Transport weiter erhöhen. Sie ist keine gemessene Rate des STM32, der verwendeten Mikrocontrollerfamilie. Vorverarbeitung kann die Übertragungsmenge verkleinern, kostet aber selbst Rechenzeit und muss in den Zeitvertrag passen. Die Rechnung begründet eine Entwurfsentscheidung und keine Pflicht, sämtliche Rohdaten immer zu unterdrücken.

Folie 12 / 50

Was am Edge übertragen werden kann

Option	Nutzen	offene Frage
Rohdaten	spätere freie Auswertung	Datenrate und Puffer
Ausschnitte	relevante Ereignisse erhalten	Auswahlkriterium
Merkmale	geringe Datenmenge	Information ausreichend?
Zustandsmeldung	einfacher Überblick	Fehler und Alter erkennbar?

Der lokale Fristpfad bleibt unabhängig vom Telemetrievertrag.

Notizen 12 / 50

Ein Merkmal ist eine aus Rohdaten berechnete zusammenfassende Größe, etwa Mittelwert oder Spitze. Ein Ausschnitt hält ausgewählte Rohdaten fest. Eine Zustandsmeldung gibt einen interpretierten Zustand samt Gültigkeit und Alter weiter. Edge heißt Verarbeitung nahe an der Quelle. Keine dieser Optionen ist allein durch den Begriff Edge vorgeschrieben.
Wer ausschließlich einen Mittelwert überträgt, kann später möglicherweise keine kurze Störung mehr rekonstruieren. Wer alle Rohdaten überträgt, braucht genügend Transportkapazität und Speicher. Beide Entscheidungen können sinnvoll sein, wenn ihr Vertrag passt. Der Capstone verwendet einen kleinen Filter als Beispiel für lokale Verarbeitung. Seine Latest-Value-Mailbox erlaubt bereits vor dem Filter Auslassungen; der Filter erhält also nicht automatisch jede ursprüngliche Probe. Das muss zur fachlichen Auswertung passen. Der Ort der Verarbeitung sagt zudem noch nichts darüber, welcher der beiden Kerne die jeweilige Aufgabe übernimmt. Diese zweite Frage beantwortet die asymmetrische Rollenverteilung.

Folie 13 / 50

AMP: verschiedene Rollen, gemeinsame Hardware

Die Aufgabenverteilung ist eine Entwurfsgrenze, keine zeitliche Mauer.



Notizen 13 / 50

AMP bedeutet Asymmetric Multiprocessing, asymmetrische Verarbeitung mit getrennten Programmen und Rollen. M4 ist der Cortex-M4 mit FreeRTOS, dem Echtzeitkernel. M7 ist der Cortex-M7 ohne Kernel. Im Diagramm überträgt die Mailbox den Anwendungsdatensatz vom erfassenden zum auswertenden Kern. Die unteren ungerichteten Linien kennzeichnen gemeinsam genutzte Hardwarebereiche wie Bus, Taktverwaltung und Reset. Sie sind keine zusätzlichen Nachrichtenpfeile.
Edge beantwortet den Ort der Verarbeitung, AMP die Rollenverteilung. Beide können gleichzeitig gelten. Ein getrennter Speichervertrag beseitigt keine Businterferenz. Zusätzliche Last auf dem M7 kann daher Zeitverhalten auf dem M4 beeinflussen. Ein System-Reset betrifft ebenfalls beide Programme nach dem vorgesehenen Startvertrag. Die zwei unterschiedlichen Cortex-Kerne rechnen hier keine gleiche Funktion im Lockstep und sind nicht automatisch gegenseitige Sicherheitsersatzkanäle. Ihre Aufteilung ist zunächst eine nachvollziehbare Arbeitsorganisation, die mit eigener Zeit- und Fehlersemantik ergänzt werden muss.

Folie 14 / 50

Fail-Safe und Fail-Operational benennen das Ziel

Muster	nach einem benannten Fehler
Fail-Safe	definierter sicherer Zustand
Fail-Operational	definierte Restfunktion bleibt verfügbar

Dazu gehören Fehlerannahme, Reaktionszeit und zulässige Dauer.
Ein Reset ist nur mit passendem Ausgangs- und Startkonzept eine sichere Reaktion.

Notizen 14 / 50

Fail-Safe führt bei einem betrachteten Fehler in einen definierten sicheren Zustand. Fail-Operational erhält nach einem betrachteten Fehler eine benannte Funktion, gegebenenfalls eingeschränkt und zeitlich begrenzt. Safety ist der Schutz vor unvertretbaren Folgen einer Fehlfunktion. Ohne Beschreibung der Anwendung, des Fehlers und der benötigten Reaktion sagen diese Wörter wenig aus.
Abschalten ist nicht in jeder Maschine ungefährlich: Eine frei fallende Last könnte gerade durch das Ausschalten gefährlich werden. Unser Zahlendemonstrator hat keinen solchen Aktor. Eine ungültige Ausgabe wird als ungültig behandelt; es wird kein erfundener Antrieb freigegeben. Auch ein Watchdog-Reset braucht das aus Vorlesung 8 bekannte Pegelkonzept während Reset und Boot. MPU, Memory

Notizen, Fortsetzung 14 / 50

Protection Unit, und HSEM, Hardware Semaphore, beantworten andere Schutzfragen. Die beiden Kerne des Demonstrators allein machen aus dem Entwurf keine Fail-Operational-Architektur. Ein Weiterbetrieb nach Fehlern benötigt passende Diagnose und häufig zusätzliche Ressourcen.

Folie 15 / 50

Redundanz: Vergleich, Umschaltung oder Abstimmung

Muster	Leistung	zusätzliche Voraussetzung
zwei Kanäle	Abweichung erkennen	Diagnose des richtigen Kanals
Active/Standby	auf Ersatz umschalten	gültiger aktueller Ersatzstand
2oo3	einen Abweichler überstimmen	passende Fehler- und Auswahlannahmen

Gemeinsame Ursache und fehlerhafter Entscheider bleiben mögliche Fehler.

Notizen 15 / 50

Redundanz bedeutet zusätzliche Ressourcen für eine benannte Aufgabe. Active ist der aktuell wirksame Kanal, Standby der vorbereitete Ersatz. Ein Zweikanalvergleich liefert bei unterschiedlichen Ergebnissen zunächst nur ein Abweichungszeichen. Ohne weitere Diagnose ist nicht bekannt, welcher Wert stimmt. Eine Umschaltung hilft nur, wenn der Ersatzkanal rechtzeitig verfügbar ist und einen gültigen ausreichend aktuellen Zustand hat.

2oo3 bedeutet two out of three, zwei von drei. Eine Abstimmung kann unter geeigneten Annahmen einen einzelnen abweichenden Kanal maskieren. Der Entscheider, Voter genannt, ist ebenfalls Teil des Fehlerkonzepts. Gleiche Anforderungsfehler, eine gemeinsame Versorgung oder derselbe falsche Algorithmus können mehrere Kanäle gleich betreffen. Unterschiedliche Chiphersteller sind deshalb nicht automatisch unterschiedliche Software. Eine Sicherheitswahrscheinlichkeit wird hier nicht aus der Zahl der Kästen berechnet. Das nächste Zahlenbeispiel macht insbesondere den Unterschied zwischen Erkennen einer Abweichung und Erkennen des richtigen Ergebnisses sichtbar.

Folie 16 / 50

Drei Werte brauchen eine Entscheidungsregel

Lehrwerte: 10,0 · 10,1 · 40,0.

- exakte Gleichheit: keine Zweiermehrheit
- bei erlaubter Nähe 0,2: erste zwei stimmen überein
- Median als festgelegte Auswahlregel: 10,1
- gemeinsame Falschwerte: auch eine Mehrheit kann falsch sein

Zwei Werte 10 und 40 verraten allein keinen richtigen Kanal.

Notizen 16 / 50

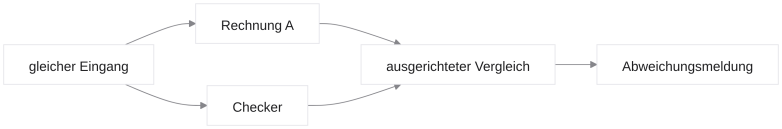
Die Werte sind frei gewählte Zahlen ohne Maßeinheit für einen Abstimmungsfall. Bei exakter Gleichheit sind alle drei verschieden. Ein Voter, der Entscheider, könnte deshalb keine exakte Zweiermehrheit finden. Erlaubt der Vertrag Abweichungen bis 0,2, passen 10,0 und 10,1 zusammen. Eine andere ausdrücklich definierte Regel ist der Median, der mittlere sortierte Wert: Nach Sortierung 10,0, 10,1, 40,0 ist das Ergebnis 10,1.

Weder Nähe noch Median beweisen, dass dieser Wert die wirkliche physische Größe trifft. Die ersten zwei Kanäle könnten dieselbe falsche Grundlage haben. Bei nur zwei Werten 10 und 40 ist die Abweichung sichtbar, aber die richtige Seite bleibt ohne zusätzliche Information unbekannt. Die Entscheidung muss zur Art des Signals passen. Für binäre Schutzanforderungen kann eine Mehrheitsregel anders aussehen als für kontinuierliche Werte. Diese Betrachtung verhindert, dass jede Wiederholung einer Rechnung als automatisch richtiger Ersatz interpretiert wird.

Folie 17 / 50

Lockstep vergleicht dieselbe Rechnung

Allgemeines Prinzip, keine Registerkonfiguration des H755.



Notizen 17 / 50

Lockstep bezeichnet die kontrollierte Ausführung derselben Funktion mit einem zweiten prüfenden Rechenpfad. Ein Checker ist dieser Prüfer. In Varianten mit zeitversetzter Ausführung werden die Ergebnisse vor dem Vergleich zeitlich ausgerichtet. Das Diagramm zeigt links denselben Eingang für beide Pfade und rechts den Vergleich mit einer möglichen Abweichungsmeldung. Die Pfeile beschreiben Datenabhängigkeiten, keine Hardwaretaktflanken oder garantierten Verzögerungswerte.

Eine Abweichung sagt zunächst, dass die verglichenen Ergebnisse nicht zusammenpassen. Sie liefert nicht automatisch ein richtiges Ersatzergebnis. Ein gemeinsamer systematischer Fehler kann beide Rechnungen gleich falsch machen. Lockstep allein ist daher keine Zusage einer Restfunktion. H755 bezeichnet hier

Notizen, Fortsetzung 17 / 50

den STM32H755 mit Cortex-M4 und Cortex-M7. Im Kurs erhalten diese Kerne verschiedene Aufgaben; sie führen dieses Lockstep-Prinzip nicht aus. Die konkrete Diagnose- und Reaktionskonfiguration einer Lockstep-Produktfamilie wäre ein eigener Gegenstand und wird nicht auf die vorhandene Plattform übertragen.

Folie 18 / 50

Gleicher Funktionsname ist noch keine Schnittstelle

Ein Sensorport muss mehr sagen als „Wert lesen“:

- Datentyp und Einheit
- Skalierung und gültiger Wertebereich
- Bedeutung von Alter und Ungültigkeit
- Aktualisierung, Auslassungen und Zugriff
- Zeit- und Fehlervertrag

Port ist hier ein Softwareanschluss.

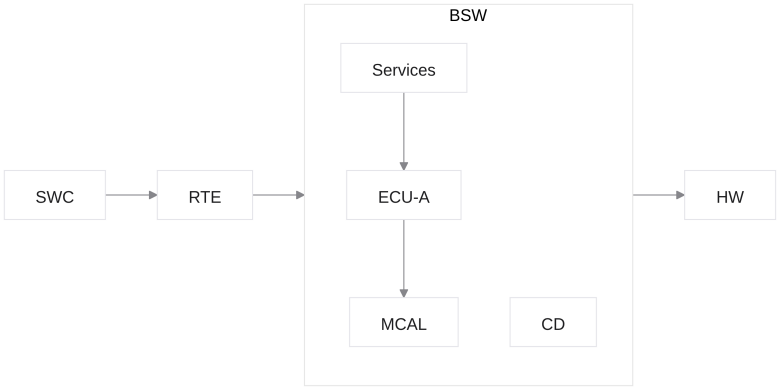
Notizen 18 / 50

Ein Port ist ein definierter Anschluss einer Softwarekomponente, kein bestimmter digitaler Pin. Ein Funktionsname wie read_sensor legt allein weder Einheit noch Alter eines Werts fest. Zehn kann zehn Volt, ein Rohcode oder ein bereits skalierter Temperaturwert bedeuten. Ebenso kann ein unveränderter Wert einen gesunden ruhigen Eingang oder eine veraltete Nachricht darstellen.
Eine Schnittstelle beschreibt deshalb Datentyp, Einheit, Skalierung, Wertebereich, Gültigkeit und Aktualisierung. Dazu kommen die Zeit- und Fehlerbedingungen. Im Capstone ist der Nutzwert ein synthetischer dimensionsloser Code zwischen null und 1023. Die Ereignis- und Publikationsnummern beschreiben seine Herkunft. Ein Zeitstempel gehört zu einer festgelegten lokalen Uhr und wird nicht beliebig auf dem anderen Kern subtrahiert. Diese kleine Schnittstelle zeigt dasselbe Entwurfsproblem, das größere Standards über viele Komponenten und Steuergeräte hinweg organisieren. Der folgende Exkurs ordnet deren Ebenen ein.

Folie 19 / 50

AUTOSAR Classic: Schichten

Schichtenbild · Labor ohne AUTOSAR-Stack.



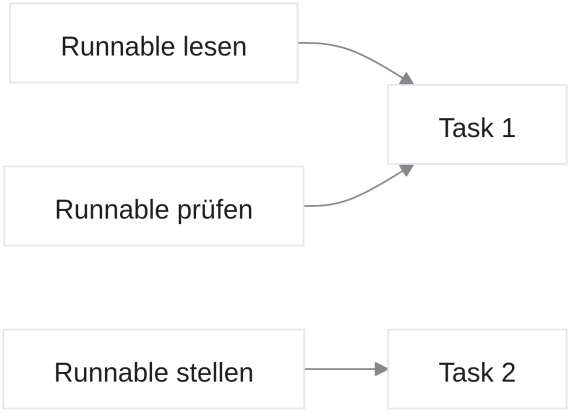
Notizen 19 / 50

AUTOSAR heißt AUTomotive Open System ARchitecture und bezeichnet Standards für Fahrzeugsoftware. Classic Platform organisiert eine Steuergerätearchitektur. SWC heißt Software Component, fachliche Softwarekomponente. RTE heißt Runtime Environment, die konkret konfigurierte Laufzeitumgebung zwischen Anwendung und Basissoftware. BSW bedeutet Basic Software. ECU heißt Electronic Control Unit, Steuergerät. MCAL heißt Microcontroller Abstraction Layer, die hardwarenahe Abstraktionsschicht. ECU-A bezeichnet im Bild die ECU-Abstraktion, CD den Complex Driver und HW die Hardware. Ein Complex Driver ist ein besonderer hardwarenaher Pfad innerhalb der BSW.
Das Diagramm liest sich für die Hauptschichten von links nach rechts: Anwendung, RTE, umrandete BSW und Hardware. Innerhalb der BSW sind Services, ECU-Abstraktion und MCAL als Teilfolge angeordnet. Services, ECU-Abstraktion, MCAL und Complex Driver liegen innerhalb dieser Umrandung. Hardware steht außerhalb rechts. Die Pfeile zeigen mögliche Beziehungen; nicht jeder Aufruf muss seriell alle Kästen durchlaufen. Der Complex Driver ist keine SWC oberhalb der RTE. Die STM32-HAL, Hardware Abstraction Layer, hat eine Treiberrolle im Labor, ist dadurch aber kein nachgewiesener AUTOSAR-MCAL-Stack. Der Nutzen des Exkurses ist die klare Zuordnung von Verantwortung und Schnittstellen.

Folie 20 / 50

Eine Softwarekomponente ist keine Task

Fiktive Zuordnung von ausführbaren Einheiten, kein Schedulingdiagramm.



Notizen 20 / 50

Ein Runnable ist eine ausführbare Einheit innerhalb einer AUTOSAR-Softwarekomponente. AUTOSAR steht für AUTomotive Open System ARchitecture. Eine Task ist dagegen ein Ausführungspfad des Betriebssystems. Das Zuordnungsdiagramm zeigt von links nach rechts drei fiktive Runnables auf zwei Tasks: lesen und prüfen werden Task eins zugeordnet, stellen Task zwei. Die Pfeile sind Zuordnungen. Sie zeigen keine Zeitfolge, Priorität oder Präemption.
Eine SWC, Software Component, kann mehrere Runnables enthalten. Deren Abbildung auf Tasks und ihre Auslöseereignisse gehören zur Konfiguration. Zwei Runnables in derselben Task teilen deren Ablauf und müssen zusammen in den Zeitvertrag passen. VFB bedeutet Virtual Functional Bus, das abstrakte Kommunikationsbild. RTE bedeutet Runtime Environment, dessen konkrete Realisierung für ein Steuergerät. Ein gleich benannter Datenport kann lokal kurz oder über ein Netzwerk deutlich verzögert realisiert sein. Daher bleibt die Antwortzeitanalyse nötig, auch wenn die logische Komponentenbeschreibung unverändert aussieht.

Folie 21 / 50

Stackelemente und Bytes unterscheiden

Lehrannahme: ein `StackType_t`-Element hat 4 Byte.

$$256 \text{ Elemente} \cdot 4 \text{ Byte} = 1024 \text{ Byte}$$

Schnittstelle	Einheit
FreeRTOS `xTaskCreateStatic`	`StackType_t`-Elemente
CMSIS-RTOS2 `stack_size`	Byte

Statische Objekterzeugung beweist weder ausreichenden Stack noch die Deadline.

Notizen 21 / 50

Der Stack ist der Speicher eines Ausführungspfads für gesicherte Zustände und lokale Daten. FreeRTOS ist der bekannte Echtzeitkernel. xTaskCreateStatic erzeugt eine Task mit vom Aufrufer bereitgestelltem Speicher und erwartet ihre Stacktiefe in StackType_t-Elementen. Im angenommenen Port hat ein solches Element vier Byte. Daher entsprechen 256 Elemente 1024 Byte.
CMSIS-RTOS2 ist die Echtzeitkernel-Schnittstelle des Common Microcontroller Software Interface Standard in Version zwei. Ihr Feld stack_size im Typ osThreadAttr_t wird dagegen in Byte angegeben. Die Zahl 256 würde dort 256 Byte bedeuten, nicht denselben 1024-Byte-Stack. Port und Adapter werden im echten Projekt geprüft. Statische Konfiguration und statische Speicherbereitstellung sind ebenfalls verschiedene Aussagen. osThreadNew ohne eigene Stack- und Verwaltungsbereiche beweist keine statische Allokation. Ein korrekt bereitgestellter Bereich beweist noch nicht, dass er für alle Pfade groß genug ist, oder dass die Task ihre geforderte Abschlussfrist, die Deadline, einhält.

Folie 22 / 50

Classic und Adaptive bleiben verschiedene Ebenen

Vergleich	Classic	Adaptive
Schwerpunkt	konfigurierte Steuergeräte	größere Diensteplattformen
Schnittstellenbild	RTE und BSW	ARA-Dienste
Betriebssystembezug	Classic-OS	spezifizierte POSIX-Bezüge

FreeRTOS ist ein Kernel. AUTOSAR ist ein Standardsatz.

Notizen 22 / 50

AUTOSAR heißt AUTomotive Open System ARchitecture. Die Classic Platform beschreibt das zuvor gezeigte Steuergerätebild. RTE bedeutet Runtime Environment, BSW Basic Software und OS Operating System, Betriebssystem. Die Adaptive Platform ist eine andere Plattformlinie. ARA steht für AUTOSAR Runtime for Adaptive Applications, deren Laufzeitdienste. POSIX bedeutet Portable Operating System Interface und bezeichnet standardisierte Betriebssystemschnittstellen.
Die Tabelle vergleicht Ziel und Schnittstellenebene, keine Rangliste. Adaptive ist nicht einfach Classic mit Linux; ein bestimmtes Betriebssystem allein definiert die Plattform nicht. OTA bedeutet Over-the-Air-Update, eine drahtlose oder über ein Netz übertragene Aktualisierung. Das ist eine Funktion, die nicht ausschließlich Adaptive vorbehalten ist. FreeRTOS kann statische Objekte unterstützen, wird dadurch aber nicht zum Classic-Betriebssystem. Unser STM32-Demonstrator ist keine Umsetzung eines der beiden AUTOSAR-Stacks. Für ihn übernehmen wir die Forderung nach eindeutigen Schnittstellen und einer nachvollziehbaren Konfiguration.

Folie 23 / 50

Was der kleine Capstone davon übernimmt

- eine schmale und vollständige Datenschnittstelle
- klare Eigentümer für Peripherie und Speicher
- kurze gemeinsame Abschnitte
- getrennte Nutzdaten- und Gesundheitsnachweise
- dokumentierte Zeit- und Fehlerbedingungen

Ein zusätzlicher Software-Layer ersetzt keinen dieser Verträge.

Notizen 23 / 50

Ein Layer ist eine Softwareebene mit benannten Aufgaben und Schnittstellen. Der Kurs übernimmt die Entwurfsprinzipien, implementiert aber keinen AUTOSAR-Stack; AUTOSAR bedeutet AUTomotive Open System ARchitecture. Für den Demonstrator ist ein kleiner selbst nachvollziehbarer Vertrag didaktisch passend. Seine Initialisierung, Übergänge und Zähler sind vollständig überschaubar.
Die Eigentümerfrage betrifft auch Peripherie: Ein Timer wird nur auf dem erfassenden Kern eingerichtet und gestartet. Eine gemeinsame Speicherregion erhält genau einen Initialisierer. Das Hardware-Schloss umschließt nur den vollständigen Datensatz. Die Datenrate sagt nicht automatisch etwas über die Gesundheit eines Workers aus. Ein unveränderter gültiger Sensorwert und ein hängender Ausführungspfad bleiben unterschiedliche Fälle. Die zweite Hälfte setzt diese Prinzipien als konkrete Ereigniskette um. Dazu werden zuerst Eingabe, Annahme, Veröffentlichung und Kopie voneinander abgegrenzt. Erst danach lassen sich Zeitgrößen und Abnahmestatus sinnvoll zuordnen.

Folie 24 / 50

Die Ereigniskette des Demonstrators

M4 besitzt die Queue · M7 filtert eine lokale Kopie.

Notizen 24 / 50

M4 ist der Cortex-M4 mit FreeRTOS, dem Echtzeitkernel. M7 ist der Cortex-M7 ohne Kernel. HSEM bedeutet Hardware Semaphore. Das Datenflussdiagramm liest sich von links nach rechts. Die Ereignisroutine des M4 übergibt eine Nachricht an die lokale Queue mit acht Plätzen. Der Worker entnimmt sie und veröffentlicht einen vollständigen Datensatz unter dem Hardware-Schloss. Der M7 erwirbt dasselbe Schloss nur zum Kopieren und bestätigt die kopierte Publikation. Der Filter arbeitet danach außerhalb. Die Queue ist kein gemeinsam benutztes Kernelobjekt des M7. Nur die Mailbox liegt im gemeinsamen Speicher. Der Normallauf erzeugt 20 synthetische Zahlen mit nominal zehn Millisekunden Abstand. Ein zweiter Lauf erzeugt zehn kontrollierte Puts vor dem ersten Get und prüft die Queuekapazität. Put und Get bedeuten Einfügen und Entnehmen. Der Programmierkern bleibt endlich und speichert seine Auswertung zunächst im privaten Arbeitsspeicher. Eine serielle Ausgabe kann danach bewusst als zusätzliche Last aktiviert werden.

Folie 25 / 50

Normallast, Burst und Sättigung unterscheiden

Versuch	Eingabe	Aussage
normal	20 Ereignisse, nominal 10 ms Abstand	fortlaufende Verarbeitung
Burst-Modell	zehn Ereignisse bei 0…9 ms	Drops hängen von Entnahmen ab
Sättigung	zehn Puts ohne Get, Queue zunächst leer	genau 8 Annahmen, 2 Drops

Zehn physische Flanken sind nicht automatisch zehn Software-Puts.

Notizen 25 / 50

Ein Burst ist eine kurze dichte Folge. Ein Put fügt eine Nachricht in die Queue ein, ein Get entnimmt eine. Im Normalversuch werden 20 Ereignisse mit nominal zehn Millisekunden Abstand erzeugt, also 100 pro Sekunde. Im Burst-Modell liegen zehn Ereignisse bei null bis neun Millisekunden; der Worker kann zwischen ihnen bereits entnehmen. Daher folgt aus Queue-Tiefe acht allein nicht, dass dieser Burst zwei Drops erzeugt. Ein Drop ist eine gezählte Abweisung.
Der kontrollierte Sättigungsmodus des Codes erzeugt zehn korrekte Puts aus der Task, bevor sie das

Notizen, Fortsetzung 25 / 50

erste Get aufruft. Bei leerer Queue und acht Plätzen müssen genau acht angenommen und zwei wegen fehlendem Platz abgewiesen werden. Dieser Modus prüft die Kapazitätslogik, keine physische Interruptlatenz. Hardware-Pending, der gespeicherte Zustand einer Unterbrechungsquelle, kann mehrere Flanken zusammenfassen. Ein Taster hat außerdem Entprellung als eigenen Vertrag. Der Demo-Timer wird deshalb nicht stillschweigend durch einen ungeprüften Tasterburst ersetzt.

Folie 26 / 50

In der Ereignisroutine wird nicht blockiert

Aus dem M4-Modul · Queue existiert vor der Quellenfreigabe.

```
osStatus_t status = osMessageQueuePut(queue, &event, 0u, 0u);
if (status == osOK) {
    ++lab09_stats.accepted;
} else if (status == osErrorResource) {
    ++lab09_stats.queue_drops;
} else {
    ++lab09_stats.put_errors;
}
```

Kein HSEM, kein Filter und keine Ausgabe in dieser Routine.

Notizen 26 / 50

M4 ist der Cortex-M4. Der C-Ausschnitt verwendet CMSIS-RTOS2, die Echtzeitkernel-Schnittstelle des Common Microcontroller Software Interface Standard in Version zwei. osMessageQueuePut fügt die Nachricht event in queue ein. Der erste Nullparameter ist die Nachrichtenpriorität, der zweite der Timeout null: Der Aufruf wartet nicht auf einen freien Platz. osOK bedeutet erfolgreich angenommen. osErrorResource bedeutet im gültigen Queuevertrag fehlenden Platz; andere Rückgaben werden separat in put_errors gezählt.
accepted zählt Annahmen, queue_drops die wegen voller Queue abgewiesenen Ereignisse. Die Routine erhält vor jedem Put eine Ereignisnummer und einen lokalen Zeitstempel. ISR bedeutet Interrupt Service Routine, Interruptroutine. Für ISR-Aufrufe muss die numerische NVIC-Priorität zur Systemaufrufgrenze des FreeRTOS-Ports passen; NVIC ist der Nested Vectored Interrupt Controller. Eine unzulässige Priorität kann einen Kernelassert auslösen und ist nicht automatisch eine gezählte Rückgabe. HSEM, Hardware Semaphore, wird erst im Worker genommen. Der vollständige Code enthält die endliche Quellenfreigabe und die Behandlung des Abschlusses.

Folie 27 / 50

Die Queuebelegung lässt sich auf Papier verfolgen

Queue-Tiefe 3, zunächst leer · FIFO-Reihenfolge.

Operation	Inhalt danach	Ergebnis
Put 1, Put 2	1, 2	beide angenommen
Get	2	entnimmt 1
Put 3, Put 4	2, 3, 4	beide angenommen
Put 5	2, 3, 4	voller Puffer, Drop
Get	3, 4	entnimmt 2

Notizen 27 / 50

FIFO bedeutet First In, First Out: Zuerst angenommene Nachrichten werden zuerst entnommen. Put bezeichnet Einfügen, Get Entnehmen und Drop die hier wegen voller Queue abgewiesene Nachricht. Das Modell benutzt drei Plätze und gleich priorisierte Nachrichten. Es beginnt leer. Nach den ersten beiden Puts enthält es eins und zwei. Das erste Get entnimmt eins, sodass zwei bleibt. Die nächsten Puts füllen mit drei und vier auf. Fünf findet keinen Platz und ändert den Inhalt nicht. Das letzte Get entnimmt zwei. Insgesamt wurden fünf Eingaben versucht, vier angenommen und eine abgewiesen. Zwei angenommene Nachrichten wurden bereits entnommen und zwei liegen noch in der Queue. Die Queue selbst kennt keine Mailboxkopie und keine Filterausgabe. Ein voller Puffer bedeutet auch nicht, dass alle früheren Nachrichten bereits verarbeitet sind. Dieses einfache Modell wird in der Übung mit acht Plätzen und verschiedenen Entnahmefolgen erweitert. Seine Bilanz trennt Eingang und Weiterverarbeitung.

Folie 28 / 50

Erste Bilanz: Eingang und Worker

$$I = A + D_q + E_{\text{put}}$$

$$A = Q + F + P + D_{\text{task}}$$

Größe	Bedeutung
$I, A, D_q, E_{\text{put}}$	Eingaben, Annahmen, Queue-Drops, sonstige Put-Fehler
Q, F	noch in Queue, gerade in Bearbeitung
P, D_{task}	publiziert, nach Entnahme verworfen

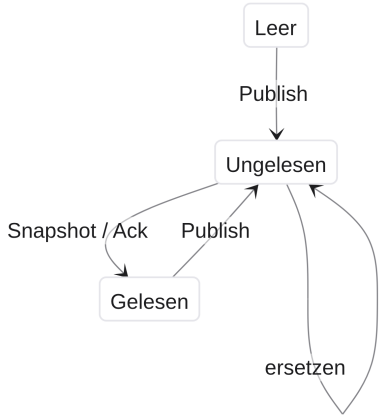
Notizen 28 / 50

I zählt die Softwareeingaben dieser Ereignisroutine, nicht automatisch jede physische Flanke. A wurde in die Queue angenommen, D q wegen voller Queue abgewiesen, E put aus einem anderen zurückgegebenen Grund abgewiesen. Die erste Summe zerlegt daher alle abgeschlossenen Putversuche. Q bezeichnet noch enthaltene Nachrichten, F entnommene noch nicht abgeschlossene Bearbeitungen. P zählt erfolgreiche Publikationen. D task zählt nach Entnahme verworfene Nachrichten, etwa einen ungültigen Nutzwert oder einen begrenzt erfolglosen Schlosserwerb.
Jede angenommene Nachricht steht an einem konsistenten Bilanzpunkt genau in einer dieser vier Gruppen. Ein beliebiges Live-Lesen mehrerer Zähler während ihres Übergangs ist kein atomarer Schnappschuss. Der endliche Demonstrator prüft daher nach gestoppter Quelle und Abschluss beider Kerne; dann sind Q und F null. Im Sättigungsbeispiel ohne nachträglichen Verwurf gelten I gleich zehn, A gleich acht, D q gleich zwei und P gleich acht. Was der andere Kern davon sieht, kommt erst in die zweite Bilanz.

Folie 29 / 50

Latest-Value und FIFO

Ein ungelesener Stand darf ersetzt werden.



Notizen 29 / 50

Latest-Value bedeutet der jeweils neueste Stand. FIFO bedeutet First In, First Out, die Entnahmereihenfolge der lokalen Queue. Im Zustandsautomaten wird die Mailbox von oben nach unten erstmals durch Publish, Veröffentlichung, ungelesen. Eine weitere Publikation kann diesen Stand ersetzen; O ist der Zähler solcher Ersetzungen. Snapshot heißt lokale vollständige Kopie, Ack Acknowledgement, Bestätigung. Die Kopie und Bestätigung machen den aktuellen Stand gelesen. Eine spätere Publikation ist wieder ungelesen.
Der Automat beschreibt Nutzdatenzustände, keine Taskzustände oder zeitliche Zusagen. Eine zweite Kopie desselben bereits bestätigten Stands zählt nicht als neue Beobachtung. Latest-Value passt zu einer Anwendung, die den aktuellen Stand braucht. Ein Algorithmus, der jede ursprüngliche Probe integrieren muss, braucht dagegen eine andere Transportstrategie oder zusätzliche Informationen. Der Filter im Demonstrator arbeitet bewusst auf der Folge der tatsächlich kopierten Stände. Daher ist eine Sequenzlücke sichtbar, aber nicht automatisch ein Protokollfehler. Ihre Herkunft muss mit den beiden Bilanzebenen getrennt werden.

Folie 30 / 50

Der Mailbox-Datensatz hat eindeutige Felder

Neues Layout in `code/lab09` · ersetzt die früheren Shared-Objekte.

Feld	Vertrag
`magic / mode / valid`	Layoutkennung, Eingabemodus, gültiger Stand
`event_seq / pub_seq`	Eingabenummer / Publikationsnummer
`event_tick / payload`	M4-lokaler Zeitpunkt / synthetischer Code
`ack_pub / overwritten / snapshots`	bestätigte Nummer / O / S
`producer_done`	Quelle gestoppt und Worker fertig

44 Byte Struktur, 64 Byte gepolsterte Section.

Notizen 30 / 50

M4 ist der Cortex-M4. Shared bezeichnet gemeinsam erreichbaren Speicher. magic ist die Kennung 0x49504339 des neuen Vertrags, mode der normal oder sättigend gewählte Eingabemodus und valid die Gültigkeit. event_seq wird vor dem Put vergeben; Lücken können deshalb aus Queueabweisung, Taskverwurf und Mailboxersetzung stammen. pub_seq steigt nur bei erfolgreicher Veröffentlichung. event_tick ist der lokale HAL-Zeitpunkt der Ereignisroutine; HAL bedeutet Hardware Abstraction Layer. payload ist der dimensionslose Code zwischen null und 1023.
ack_pub ist die zuletzt bestätigte Publikationsnummer. overwritten und snapshots zählen O, vor erster Kopie ersetzte Stände, und S, verschieden kopierte Stände. producer_done wird erst nach Quellenstopp und vollständiger Queuebearbeitung gesetzt. Die elf 32-Bit-Wörter ergeben 44 Byte. Der Linker ergänzt auf 64 Byte bei 32-Byte-Ausrichtung. Beide Images, die Firmwares der Kerne, legen dieses Objekt in SRAM4, der gemeinsamen Bank des Static Random Access Memory, ab 0x38000000. Alte Objekte aus Vorlesung 6 oder 8 dürfen dort nicht gleichzeitig liegen.

Folie 31 / 50

Publizieren heißt einen vollständigen Satz ändern

Unter HSEM nach bestätigtem Erwerb:

```
if (lab09_box.valid && lab09_box.ack_pub != lab09_box.pub_seq) {
    ++lab09_box.overwritten;
}
lab09_box.event_seq = event.event_seq;
lab09_box.pub_seq = event.pub_seq;
lab09_box.event_tick = event.event_tick;
lab09_box.payload = event.sample;
lab09_box.valid = 1u;
```

Danach DMB und passende Freigabe. Keine Ausgabe unter dem Schloss.

Notizen 31 / 50

HSEM heißt Hardware Semaphore, DMB Data Memory Barrier zur Ordnung von Speicherzugriffen. Der C-Ausschnitt steht innerhalb der vollständigen Publish-Funktion. event ist die lokal entnommene Nachricht mit der neu vergebenen Publikationsnummer. Vor dem Überschreiben prüft der Producer, der veröffentlichende Worker, ob der alte gültige Stand bereits bestätigt wurde. Ist ack_pub ungleich pub_seq, zählt overwritten eine erlaubte Ersetzung.
Anschließend werden Herkunftsnummer, Publikationsnummer, lokaler Ereigniszeitpunkt und Nutzwert gemeinsam geändert. valid markiert den gültigen Stand. Das Hardware-Schloss verhindert ein gemischtes Lesen dieser Felder. Die vollständige Implementierung prüft das Protokoll, nimmt mit der Prozesskennung 0x34 und gibt mit derselben Kennung frei. Sie benutzt nach Erwerb und vor Freigabe eine Barriere. Der Cortex-M7 verwendet die andere Kennung 0x37. Sichtbarkeit kommt zusätzlich aus der nicht cachebaren Region. Ein fehlgeschlagener Erwerb darf nicht zu einem ungeschützten Teilupdate führen; im begrenzten Versuch wird die Nachricht stattdessen separat als Taskverwurf gezählt.

Folie 32 / 50

Kopieren, bestätigen, dann filtern

Lehrfolge mit Publikationsnummern:

Aktion	Mailbox	Wirkung
M4 publiziert 1, dann 2	2 ungelesen	1 ersetzt
M7 kopiert 2 und bestätigt	2 gelesen	ein neuer Snapshot
M7 liest 2 erneut	2 gelesen	kein neuer Snapshot
M4 publiziert 3	3 ungelesen	bleibt zunächst ungelesen

Am Ende: $P = 3, S = 1, O = 1, U = 1.$

Notizen 32 / 50

M4 und M7 bezeichnen die Cortex-Kerne. P zählt Publikationen, S verschieden kopierte Stände, O vor ihrer ersten Kopie ersetzte Stände und U den aktuell noch ungelesenen Stand. Die Tabelle beginnt leer. Der M4 veröffentlicht eins und anschließend zwei, bevor der M7 liest. Damit wurde eins ersetzt und O ist eins. Der M7 kopiert zwei vollständig unter dem Schloss und schreibt ack_pub gleich zwei, seine Bestätigung. S wird eins. Ein erneutes Lesen von zwei erhöht S nicht.
Danach veröffentlicht der M4 drei. Weil zwei bestätigt war, entsteht keine zusätzliche Ersetzung. Drei ist noch ungelesen, daher U gleich eins. Die Summe eins plus eins plus eins ist drei. Der Filter läuft erst nach der Freigabe auf der lokalen Kopie. Eine langsame Ausgabe kann weitere spätere Ersetzungen begünstigen, macht aber die bereits kopierten Felder nicht gemischt. Der Abschlussversuch kopiert auch den letzten Stand vor dem Ende; seine finale Bilanz hat deshalb U gleich null.

Folie 33 / 50

Mailboxbilanz: Publikation und Kopie

Unter dem gemeinsamen Schloss oder nach kontrolliertem Abschluss:

$$P = S + O + U, \qquad U \in \{0, 1\}$$

Beispiel: $P = 8$, $S = 5$, letzter Stand bestätigt.

$$8 = 5 + 3 + 0$$

Ereignislücken allein erklären die Ursache nicht.

Notizen 33 / 50

P ist die Zahl erfolgreicher Publikationen, S die Zahl erstmals kopierter verschiedener Publikationen und O

Notizen, Fortsetzung 33 / 50

die Zahl vor ihrer ersten Kopie ersetzter Publikationen. U ist null oder eins und zeigt, ob der aktuelle Stand noch ungelesen ist. Unter dem HSEM, Hardware Semaphore, werden die Felder der Mailbox gemeinsam geprüft und verändert. Die Implementierung prüft diese Invariante, eine an den Transaktionsgrenzen gültige Eigenschaft, vor ihren Änderungen.
Im Beispiel sind acht Stände publiziert. Fünf wurden verschieden kopiert und der letzte ist bestätigt. Es bleiben genau drei Ersetzungen, weil U null ist. Ein Snapshot ist ein Datensatz, keine Summe aller verarbeiteten Nachrichten. Werden die acht Publikationen aus zehn Eingaben mit zwei Queue-Drops erzeugt, ist die Eingangsabweisung eine andere Gruppe als die drei Mailboxersetzungen. Eine Ereignisnummer wird bereits vor dem Put vergeben und kann beide Ursachen widerspiegeln. Die zusätzliche Publikationsnummer und die Bestätigung ermöglichen erst diese saubere zweite Bilanz. In den 20-Ereignis-Läufen tritt kein Nummernumlauf auf.

Folie 34 / 50

Erwerbswartezeit und Haltezeit unterscheiden

Gemeinsame Modelluhr in µs:

Marke	Zeitpunkt
Bitte um das Schloss	100
bestätigter Erwerb	140
Freigabe	165

$$T_{\text{wait}} = 40\,\mu\text{s}, \qquad T_{\text{hold}} = 25\,\mu\text{s}$$

Ein Marker um den gesamten Aufrufpfad kann 65 µs umfassen.

Notizen 34 / 50

T wait ist die Erwerbswartezeit, T hold die Haltezeit. Die Modelluhr ist gemeinsam und in Mikrosekunden angegeben. Von der Bitte bei 100 bis zum bestätigten Erwerb bei 140 vergehen 40 Mikrosekunden. Von Erwerb bis Freigabe bei 165 sind es 25 Mikrosekunden. Zusammen ergeben sich 65 Mikrosekunden für das gesamte betrachtete Intervall.
HSEM bedeutet Hardware Semaphore. Der Erwerbsversuch kann scheitern und wiederholt werden. Eine kurze Haltezeit allein begrenzt deshalb nicht die gesamte Wartezeit: Andere Teilnehmer, eigene Pausen und fehlende Fairness gehören in den Vertrag. Das Lehrziel von 50 Mikrosekunden gilt für die Haltezeit je Seite. Ein GPIO, General Purpose Input/Output, ist ein digitaler Ein- oder Ausgang. Ein GPIO-Marker vor Take und nach Release kann beide Anteile enthalten und ist ohne weitere Definition kein Nachweis genau dieser Haltezeit. Zudem kosten die Marker selbst Zeit. Ein Millisekundentick kann 25 oder 50 Mikrosekunden nicht sinnvoll auflösen; die Zeitquelle muss dazu passen.

Folie 35 / 50

Ein Filter auf der tatsächlich kopierten Folge

Gleitender Mittelwert der letzten höchstens vier Kopien:

$$y_k = \left\lfloor \frac{\sum_{j=0}^{m-1} x_{k-j}}{m} \right\rfloor, \quad m = \min(k, 4)$$

kopierte Eingaben	10	14	18	22	26
Ergebnis	10	12	14	16	20

Keine Nullauffüllung am Start. Ausgelassene Publikationen sind keine Filtereingaben.

Notizen 35 / 50

x ist der kopierte synthetische Nutzwert, y das gefilterte Ergebnis und k der ab eins gezählte neue Snapshot. m ist die Zahl bereits verfügbarer Fensterwerte, höchstens vier. Die Bodenklammer bedeutet Abrunden der nichtnegativen Division auf eine ganze Zahl. Am Anfang wird durch die wirklich vorhandene Zahl dividiert, nicht durch vier mit hinzugefügten Nullen.
Bei zehn ist der Mittelwert zehn. Zehn plus 14 durch zwei ist zwölf. Mit 18 ergeben 42 durch drei den Wert 14. Mit 22 ergeben 64 durch vier den Wert 16. Der fünfte Wert ersetzt zehn im Fenster; 14 plus 18 plus 22 plus 26 ist 80, durch vier ergibt 20. Das C-Modul hält einen Ringpuffer mit vier Werten und aktualisiert seine Summe außerhalb des HSEM, Hardware Semaphore. Mit Nutzwerten höchstens 1023 bleibt die Summe höchstens 4092 und passt in den 32-Bit-Typ. Der Filter sieht ausschließlich tatsächlich kopierte Stände. Soll er jede ursprüngliche Probe berücksichtigen, wäre Latest-Value, der neueste Stand, kein ausreichender Transportvertrag.

Folie 36 / 50

Serielle Ausgabe ist ein eigener Zeitpfad

Lehrpaket `P=0001\r\n`: acht ASCII-Zeichen.

$$8 \cdot 10 = 80 \text{ Bit}, \qquad \frac{80}{115200} = 0{,}69444 \text{ ms}$$

- 8N1: Startbit, 8 Datenbits, keine Parität, Stopbit
- reine Leitungszeit, keine CPU-Zeit
- tatsächliche Demozeilen haben variable Länge
- UART-Ausgabe stets außerhalb des HSEM

Notizen 36 / 50

UART heißt Universal Asynchronous Receiver/Transmitter, serielle Schnittstelle. ASCII bedeutet American Standard Code for Information Interchange und ist die Zeichenkodierung des Lehrpakets. P gleich 0001 enthält sechs Zeichen; der Wagenrücklauf und der Zeilenumbruch, als Backslash r und Backslash n dargestellt, ergeben zwei weitere. Unter 8N1 werden pro Zeichen ein Startbit, acht Datenbits und ein Stopbit übertragen. Keine Parität bedeutet, dass kein Paritätsbit dazukommt. Acht Zeichen brauchen 80 Bitzeiten.
Bei 115200 Bit pro Sekunde sind das rund 0,69444 Millisekunden reine Leitungszeit. CPU bedeutet Central Processing Unit, Prozessorkern. Die Leitungszeit ist nicht die eigene Prozessorarbeit. Ein blockierender Sender kann diese Zeit dennoch zum verstrichenen Ausgabeintervall hinzufügen. HSEM bedeutet Hardware Semaphore; es wird vorher freigegeben. Die optionalen Demozeilen enthalten mehrere Felder und haben eine andere tatsächliche Länge. Im Basislauf bleiben Ergebnisse im privaten Arbeitsspeicher. Eine aktivierte Ausgabe verändert die Snapshotrate und kann erlaubte Mailboxersetzungen erhöhen.

Folie 37 / 50

Zeitmarken brauchen dieselbe Kennung

Marke	festgelegtes Ereignis
t_1	Beginn der M4-Ereignisroutine
t_2	erfolgreicher Put dieser Nachricht beendet
t_3	Get dieser Nachricht, Bearbeitung beginnt
t_4	Veröffentlichung an definierter Freigabestelle
t_5	neue lokale M7-Kopie abgeschlossen
t_6, t_7	Filter / definierter Ausgabeabschluss

Ein lokaler Tick im Datensatz ist keine gemeinsame Uhr.

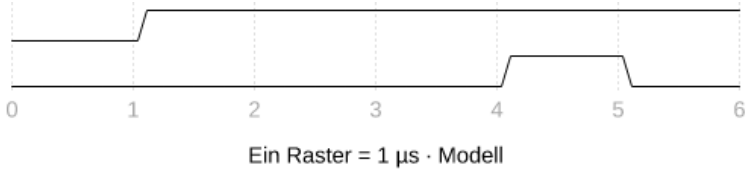
Notizen 37 / 50

M4 ist der Cortex-M4, M7 der Cortex-M7. Die Zeitpunkte t eins bis t sieben gehören jeweils zu derselben Nachricht beziehungsweise ihrer Publikation. Put bezeichnet Einfügen in die Queue, Get Entnehmen. t eins liegt am dokumentierten Beginn der Software-Ereignisroutine, nicht automatisch an der ersten physischen Flanke oder am ersten Maschinenbefehl der Exception. t vier benötigt eine präzise Marke relativ zur Hardwarefreigabe. Eine Marke nach Release kann vom anderen Kern zeitlich schon überholt werden.
Die Publikationsantwort ist t vier minus t eins, das Alter eines kopierten neuen Stands t fünf minus t eins. Für diese Inter-Core-Differenz braucht man einen gemeinsamen externen Zeitbezug, einen geeignet gemeinsam gelesenen Timer oder eine belegte Korrelation. Zwei lokale DWT-Zähler, Data Watchpoint and Trace, sind nicht automatisch synchron. Der gespeicherte event_tick stammt aus der lokalen HAL-Millisekundenuhr des M4; HAL bedeutet Hardware Abstraction Layer. Er hilft bei der Herkunftsdiagnose, beweist aber keine feine Haltezeit und darf nicht von einer beliebigen M7-Uhr subtrahiert werden.

Folie 38 / 50

Physische Flanke und Softwaremarke trennen

Synthetische digitale Spuren · oben Eingang, unten Marker der Ereignisroutine.



Ein Raster = 1 µs · Modell

Flanke bei 1 µs, Marker bei 4 µs → Abstand 3 µs.

Notizen 38 / 50

Die obere digitale Spur stellt den Eingang dar, die untere den GPIO-Marker, General Purpose Input/Output, am dokumentierten Beginn der Ereignisroutine. Die waagerechte Achse hat ein Mikrosekundenraster. Der Eingang steigt bei eins, der Marker bei vier und fällt bei fünf. Der Abstand der steigenden Flanken ist drei Mikrosekunden; die Markerbreite ist eine Mikrosekunde und eine andere Größe. Beide Zahlen sind ein synthetisches Modell, keine Messung des Boards.
Zwischen physischer Flanke und Softwaremarke können Peripherieerkennung, Interruptannahme, Eintrittscode und ein Framework-Handler liegen. Diese Verzögerung gehört nicht automatisch zu einer erst an der Marke beginnenden Publikationsantwort. Ein externer gemeinsamer Recorder kann beide Flanken zeitlich beziehen. Der Marker ist selbst zusätzliche Arbeit und bildet keine analoge Flankensteilheit ab. Wiederholte Eingaben brauchen zudem eine eindeutige Zuordnung. Fehlt ein Ereignis im Trace, der geordneten Aufzeichnung, darf man dafür weder null Verzögerung noch eine erfundene interpolierte Zeit einsetzen.

Folie 39 / 50

Publikationsantwort und Spanne auswerten

Gemeinsame Modelluhr, korrelierte Ereignisse, alle Zeiten in µs.

Ereignis	t_1	t_4	R_{pub}
1	100	600	500
2	10000	10750	750
3	20000	20900	900

$$R_{\text{pub}} = t_4 - t_1, \quad J_R = 900 - 500 = 400\,\text{µs}$$

Notizen 39 / 50

R pub ist die Publikationsantwort vom Beginn der M4-Ereignisroutine t eins bis zur definierten Veröffentlichung t vier. M4 bezeichnet den Cortex-M4. J R ist die Spanne der drei Antworten. Die erste Differenz ist 600 minus 100 gleich 500 Mikrosekunden, die zweite 10750 minus 10000 gleich 750 und die dritte 20900 minus 20000 gleich 900. Die Spanne ist 900 minus 500 gleich 400 Mikrosekunden.
Im Lehrmodell sind die Kennungen vollständig und die Zeitbasis gemeinsam. Die drei Stichproben erfüllen jeweils das Ziel einer Millisekunde, denn 900 Mikrosekunden sind 0,90 Millisekunden. Eine reale Aufnahme bräuchte diese Voraussetzungen ebenfalls und müsste die Zahl gültiger Paare nennen. Abgewiesene Queueeingaben haben keinen Publikationsabschluss und erhalten keine künstliche Antwortzeit. Geplante Publikationen ohne gültige Endmarke bleiben in der Abdeckung sichtbar. Das beobachtete Maximum bleibt ein Stichprobenergebnis und kein allgemeiner Zeitbeweis. Die zweite Zeitgröße betrachtet nun die tatsächlich kopierten Publikationen.

Folie 40 / 50

Datenalter ist keine Vollständigkeitsbilanz

Gemeinsame Modelluhr in ms:

Publikation	t_1	t_5	Bewertung
1	0	—	vor Kopie ersetzt
2	10	45	Alter 35 ms, Ziel erfüllt
3	20	80	Alter 60 ms, Ziel verletzt

$$A_{\text{snap}} = t_5 - t_1$$

Ersetzt heißt kein Alterspaar; es bleibt eine gezählte Auslassung.

Notizen 40 / 50

A snap ist das Alter eines tatsächlich neu kopierten Stands vom Beginn seiner Ereignisroutine t eins bis zum Abschluss seiner Kopie t fünf. Die erste Publikation wurde vor einer Kopie ersetzt. Für sie existiert kein t fünf und somit kein gültiges Alterspaar. Das bedeutet nicht Alter null. Sie wird stattdessen in O, der Zahl vor erster Kopie ersetzter Publikationen, gezählt.
Für die zweite Publikation ergibt 45 minus zehn ein Alter von 35 Millisekunden, innerhalb des Lehrziels 50. Die dritte ergibt 80 minus 20 gleich 60 Millisekunden und verletzt das Ziel. Diese Modellfälle unterscheiden also Ersetzung von einer wirklich beobachteten Fristverletzung. Ein gültiger alter Datensatz kann bei Sensorstille weiter dieselbe Zahl enthalten. Er darf nicht allein wegen unveränderter Publikationsnummer als frischer Wert gelten. Eine konkrete Anwendung muss das zulässige Alter und den Umgang mit einem veralteten Stand festlegen. Die Frische des Ausführungspfads ist wiederum eine andere Frage als die Neuheit des Nutzwerts.

Folie 41 / 50

Eine Antwortzeit ohne doppelte Anteile zerlegen

Gemeinsames Lehrmodell, Zeiten in µs:

Abschnitt	Abstand
Ereignisroutine: $100 \to 110$	10
bis Get: $110 \to 300$	190
Vorbereitung: $300 \to 330$	30
Erwerbswarten: $330 \to 350$	20
Halten/Publizieren: $350 \to 370$	20

$$R_{\text{pub}} = 10 + 190 + 30 + 20 + 20 = 270\,\mu\text{s}$$

Notizen 41 / 50

Das Modell zerlegt die verstrichene Publikationsantwort R pub in direkt aneinander anschließende Intervalle. Get ist die Entnahme der betrachteten Nachricht. Das erste Intervall endet nach dem erfolgreichen Put. Das zweite umfasst alle Verzögerungen bis zum Get, einschließlich möglicher restlicher Interruptarbeit und Einplanung. Die Vorbereitung endet bei der Bitte um das Schloss. Es folgen Erwerbswarten und der tatsächliche Besitz bis zur Veröffentlichung.

Die fünf disjunkten Abstände sind zehn, 190, 30, 20 und 20 Mikrosekunden. Ihre Summe ist 270, ebenso wie 370 minus 100. Es werden keine überlappenden Intervalle zusätzlich summiert. Die gesamte Wartezeit und der darin enthaltene Erwerbsabstand dürfen beispielsweise nicht beide nochmals in dieselbe Summe eingehen. Das ist eine Modellrechnung, keine gemessene Worst-Case-Grenze. Für eine echte obere Antwortzeit müssen auch zulässige Eingabelast, Prioritäten, Maskierung, Buslast und die maximalen Einzelanteile begründet werden. Ein beobachteter schneller Normalfall ersetzt diese Annahmen nicht.

Folie 42 / 50

Nutzdatenstille ist kein Worker-Stillstand

Der Supervisor prüft frische vollständige Kontrollrunden.

Fall	Datenfolge	Gesundheitsfrage
kein neues Eingangssignal	Nummer steht	läuft die Kontrollrunde weiter?
Burst	viele Eingaben	Queue/Verarbeitung begrenzt?
M7 hängt	keine neuen Kopien	fehlt sein erforderlicher Nachweis?

WWDG-Refresh folgt der geprüften Policy, nicht jeder Eingabe.

Notizen 42 / 50

Ein Supervisor ist die überwachende Stelle. M4 ist der Cortex-M4, M7 der Cortex-M7. WWDG bedeutet Window Watchdog, Fenster-Watchdog. Refresh ist das Nachladen des Zählers. Ein gesundes System kann zeitweise keine neuen Nutzdaten erhalten. Der Worker muss trotzdem die vereinbarte Kontrollrunde oder eine andere unabhängig begründete Fortschrittsbedingung erfüllen. Ein bloß unveränderter Nutzwert ist deshalb kein Hängernachweis.
Umgekehrt kann ein weiterlaufender M4-Refresh einen ausgefallenen M7 verdecken, wenn dessen Fortschritt nicht in die Gesundheitsbedingung eingeht. Ein Burst darf nicht zu jedem Eingang einen Refresh auslösen, weil die kurzen Abstände das Hardwarefenster verletzen können. Unterschiedliche Rundenperioden benötigen die eigenen Frischefristen aus Vorlesung 8. Der endliche Programmierkern dieses Termins demonstriert die Datenkette und ihre Bilanzen; er enthält ausdrücklich keinen kompletten laufenden Supervisor. Für einen integrierten Wächter müssten der gültige Normalbetrieb, die legitime Stille, das bewusst endliche Versuchsende und der Fehlerzustand zusätzlich geregelt werden. Die Watchdogstation aus Vorlesung 8 bleibt ein separater Versuch.

Das Fenster bleibt eine Hardwarebedingung

Lehrkonfiguration WWDG2: PCLK 100 MHz, Teiler 8, Reload 127, Window 80.
$\Delta t = 4096 \cdot 8/100\,000\,000 = 0{,}32768$ ms.

Grenze	idealer Zeitpunkt nach Reload
$T = 80 \cdot$ Refresh erlaubt	15,40096 ms
$T = 64 \cdot$ EWI	20,64384 ms
$T = 63 \cdot$ Reset	20,97152 ms

18 ms ist ein geplantes Ziel, kein garantierter Ausführungstermin.

Notizen 43 / 50

WWDG heißt Window Watchdog, PCLK Peripheral Clock, hier der APB1-Bustakt; APB bedeutet Advanced Peripheral Bus. Reload ist der Nachladewert, Window die obere Refreshgrenze und T der aktuelle abwärts laufende Zählerstand. EWI bedeutet Early Wakeup Interrupt, Warnunterbrechung. Die Lehrwerte sind dieselben wie in Vorlesung 8. Ein Schritt dauert 4096 mal acht geteilt durch 100 Millionen Sekunden, also 0,32768 Millisekunden.
Bis 80 sind 47 Schritte nötig, bis 64 63 und bis 63 64. Multiplikation liefert die drei Tabellenzeiten. Hardwareseitig erlaubt ist 63 kleiner T und T höchstens 80; Gleichheit mit 80 gehört dazu. Die Tabelle nimmt den ersten Schritt einen vollen Tick nach Reload an. Die wirkliche Vorteilerphase und die Softwareverzögerung brauchen Reserve. Ein periodisches 18-Millisekunden-Ziel kann unabhängig von Eingabebursts geplant werden, ist aber ohne begrenzten Aufwachverzug kein Zeitbeweis. Der EWI ist maskierbar und kein garantierter Abschaltpfad. Diese Rechnung gehört zur Entwurfsprüfung, nicht zu vorab ausgefüllten Beobachtungszellen.

Folie 44 / 50

Kriterien C1–C4: Last und Nutzdatenzeiten

Kriterium	Lehrziel	Voraussetzung
C1	20 normale Eingaben, nominal 100 Hz	Quelle und reale Takte dokumentiert
C2	Sättigung: 8 Annahmen, 2 Queue-Drops	leer, 10 korrekte Puts, kein Get
C3	$R_{\text{pub}} \leq 1$ ms	Normallast, vorgesehene Publikationen
C4	$A_{\text{snap}} \leq 50$ ms	neue Kopien, gemeinsame Zeitbasis

Zu C3/C4 gehören Zahl gültiger Paare und die nicht erfassten Fälle.

Notizen 44 / 50

C eins bis C vier sind Kennungen von Abnahmekriterien. C eins beschreibt die endliche normale Eingabe mit 100 Ereignissen pro Sekunde. C zwei ist ein gesonderter Kapazitätstest der Queue, des lokalen Nachrichtenpuffers. Sein exakter Sollwert folgt aus den ausdrücklich gesetzten Bedingungen, nicht aus jedem beliebigen Burst. C drei prüft R pub, die Publikationsantwort vom Beginn der Ereignisroutine bis zur definierten Veröffentlichung. C vier prüft A snap, das Alter einer tatsächlich neu kopierten Publikation. C drei gilt für angenommene zur Publikation vorgesehene Nachrichten unter der Normallast. Ein absichtlich verworfener Datensatz wird bilanziert und nicht als Nullantwortzeit eingetragen. C vier braucht einen gemeinsamen Zeitbezug der Cortex-Kerne; unkorrelierte lokale Uhren reichen nicht. Auch eine vollständige Mailboxbilanz beweist diese Zeitziele nicht. Fehlende Endmarken werden als fehlende Abdeckung genannt. Die Architektur und die beobachtete Stichprobe können getrennte Status erhalten. Die übrigen Kriterien prüfen nun die Voraussetzungen dieses Zeitpfads.

Folie 45 / 50

Kriterien C5–C8: Vertrag und Prüfgrenzen

Kriterium	zu prüfen
C5	eigener HSEM, Shared-Layout, Start und Sichtbarkeit
C6	Besitzhaltezeit je Seite höchstens 50 µs
C7	Watchdogrechnung und passende Fortschrittspolicy
C8	beide Bilanzen an konsistentem Prüfpunkt

Für den endlichen Code ist C7 eine Entwurfs-/Rechenprüfung. Ein integrierter Watchdoglauf wäre eine Erweiterung.

Notizen 45 / 50

HSEM heißt Hardware Semaphore. Shared-Layout ist die Anordnung des gemeinsamen Datensatzes, mit gleichem Zielort in beiden Images, den Firmwares der Kerne. C fünf reserviert eine eigene Nutzdatenressource und lässt das Boot-Schloss separat. Es prüft außerdem den einzigen Initialisierer und die nicht cachebare Sichtbarkeit. C sechs betrifft die Haltezeit nach bestätigtem Erwerb, nicht die gesamte Erwerbswartezeit.
C sieben behandelt die Rechnung des Hardwarefensters und die Gesundheitsbedingungen. Das bereitgestellte endliche Datenmodul startet keinen Watchdog. Deshalb ist dessen Codeabschluss kein bestandener integrierter Watchdogtest. Die separate Station aus Vorlesung 8 oder ein eigens erweiterter Supervisor benötigen einen eigenen Versuch und Status. C acht prüft I gleich A plus D q plus E put, A gleich Q plus F plus P plus D task und P gleich S plus O plus U. I sind Eingaben, A Annahmen, D q Queueabweisungen und E put sonstige Putfehler. Q liegt noch im Puffer, F wird bearbeitet, P ist publiziert und D task nach Entnahme verworfen. S ist verschieden kopiert, O vorher ersetzt und U der ungelesene aktuelle Stand. Die Größen werden nach kontrolliertem Abschluss beziehungsweise innerhalb der jeweiligen Besitztransaktion gelesen. Leere Beobachtungsfelder bleiben ungeprüft.

Folie 46 / 50

Eine Testmatrix trennt Erwartung von Befund

Test	analytische Erwartung	ohne Lauf
normale Datenkette	Bilanzen und Kennungen konsistent	nicht geprüft
kontrollierte Sättigung	8 Annahmen, 2 Drops	nicht geprüft
langsamere M7-Kopie	mögliche Ersetzungen, gezählt	nicht geprüft
ungültige Messung	kein verwendbares Zeitpaar	Messung ungültig
stehender Partner	begrenzter Abbruchpfad	nicht geprüft

Notizen 46 / 50

M7 bezeichnet den Cortex-M7. Eine analytische Erwartung folgt aus Modell und Programmvertrag. Ein Befund braucht dagegen einen tatsächlich ausgeführten Versuch. Die normale Datenkette hat kein pauschales Soll von 20 Snapshots: Latest-Value erlaubt Ersetzungen, solange sie korrekt gezählt werden. Die Sättigung hat unter ihren streng kontrollierten Bedingungen die genaue Annahmezahl acht und Dropzahl zwei. Eine bewusst langsamere M7-Schleife kann weniger Stände kopieren; die Bilanzen müssen trotzdem passen.

Die Statusworte sind getrennt: nicht geprüft, bestanden unter den dokumentierten Bedingungen, verletzt oder Messung ungültig. Fehlt eine gemeinsame Zeitbasis, ist ein Zeitkriterium nicht geprüft. Eine beschädigte Aufnahme wird nicht durch null Latenz vervollständigt. Ein absichtlich angehaltener Partner ist Fehlerinjektion und kann den Abbruch prüfen, nicht die normale Publikationsfrist. Der Timeout braucht laufenden prüfenden Code und seine Zeitbasis. Er ersetzt keinen Hardwarewächter. Der Bericht nennt deshalb, welcher Pfad tatsächlich geprüft wurde und welche Annahmen offen bleiben.

Folie 47 / 50

Der Programmierteil hat zwei klare Läufe

NUCLEO-H755ZI-Q · CubeMX · VS Code mit STM32 VS Code Extension.

1. Vorbereitetes Dual-Core-CMake-Projekt weiterverwenden
2. `code/lab09` anstelle vorheriger Laborobjekte einbinden
3. normal: M4-Timer, 20 Ereignisse, M7-Pause 1 ms
4. Sättigung: 10 Puts vor Get, kein Timerstart
5. beide Kerne abschließen, dann Bilanzen und Filter prüfen

25-ms-M7-Pause und UART sind gezielte Vergleichslasten.

Notizen 47 / 50

CubeMX ist das Konfigurations- und Generierungswerkzeug von STMicroelectronics. VS Code ist der Editor, STM32 VS Code Extension die Entwicklungs- und Debugerweiterung. CMake beschreibt die beiden Buildziele. M4 und M7 sind die Cortex-Kerne. UART heißt Universal Asynchronous Receiver/Transmitter, serielle Schnittstelle. Der vollständige Anwendungscode und die Integration stehen unter code/lab09. Ein generiertes Boardprojekt ist zusätzlich Voraussetzung, kein erfundenes Ergebnis des Folienexports.
Der M7 konfiguriert den gemeinsamen Speicher bei ausgeschaltetem Datencache und initialisiert ihn vor der bestehenden Freigabe des noch wartenden M4. Beide Linkerskripte reservieren die Region und entfernen frühere Laborobjekte. Auf dem M4 existiert die Queue vor der Timerfreigabe. Der Modus wird einmal vom M7 veröffentlicht. Nach dem endlichen Lauf sind Quelle gestoppt, Queue gelöscht und Producer, der veröffentlichende Worker, fertig. Der M7 kopiert den letzten Stand und beendet die Schleife. Erst danach werden private und gemeinsame Diagnosewerte bei angehaltenen Kernen gelesen. Zusätzliche Ausgabe- oder Pausenlast verändert den Versuch und wird separat dokumentiert.

Folie 48 / 50

Papiertransfer: zwei Bilanzen gleichzeitig

Konsistenter Prüfpunkt:
$I = 12, A = 10, D_q = 2, E_{\text{put}} = 0.$

$$Q = 1,\ F = 1,\ D_{\text{task}} = 1,\ S = 4,\ U = 1$$

Bestimmen Sie P und O. Ist „Snapshotwert gleich zehn“ eine ausreichende Bilanz?
Zusatz: Wie ändert sich U, wenn der letzte Stand jetzt erstmals kopiert wird?

Notizen 48 / 50

I sind Eingaben, A Annahmen, D q Queueabweisungen und E put andere Putfehler. Die Eingangsbilanz zwölf gleich zehn plus zwei plus null ist erfüllt. Q liegt noch in der Queue, F ist in Bearbeitung und D task nach Entnahme verworfen. Aus A gleich Q plus F plus P plus D task folgt P gleich zehn minus eins minus eins minus eins, also sieben.
S sind verschieden kopierte Stände und U die aktuelle ungelesene Publikation. Aus P gleich S plus O plus U folgt O gleich sieben minus vier minus eins, also zwei. Ein Nutzwert zehn enthält keine dieser Zählinformationen und kann die Bilanz nicht ersetzen. Wird der aktuelle Stand erstmals kopiert und bestätigt, steigt S von vier auf fünf und U fällt von eins auf null. P und O bleiben sieben beziehungsweise zwei. Damit bleibt sieben gleich fünf plus zwei plus null. Der Prüfpunkt ist ausdrücklich konsistent; ein beliebiges gleichzeitiges Lesen laufender Übergangszähler wäre keine Grundlage dieser Rechnung.

Folie 49 / 50

Was eine abgeschlossene Abnahme enthalten muss

- Last, Projektstand und Messvertrag
- nachvollziehbare Rechen- und Codebefunde
- echte Beobachtungen mit Kennung und Abdeckung
- Prüfstatus je Kriterium und begründete Grenzen
- offener Wiederanlauf- und Fehlervertrag für Erweiterungen

Ein gemessenes Maximum beschreibt die gültige Stichprobe.

Notizen 49 / 50

Ein Messvertrag legt Ereignisgrenzen, Einheit, Zeitquelle und Bedingungen fest. Abdeckung beschreibt, welche erforderlichen Ereignisse und Pfade tatsächlich erfasst wurden. Die Abnahme verknüpft diese Angaben mit den Kriterien und ihren Statuswerten. Eine logische Hostprüfung, also eine Ausführung mit modellierten Hardwarefunktionen auf dem Entwicklungsrechner, kann Programmzustände und Bilanzen prüfen. Sie beweist keine Cortex-Buslatenzen oder echte Watchdogzeiten.
Auch ein erfolgreiches Ziel-Binary, die kompilierte Firmware, wäre noch keine Messung auf dem Board. Ein gültiger Boardlauf zeigt wiederum die geprüften Bedingungen, nicht alle unbekannten Pfade. WCET heißt Worst-Case Execution Time, obere Ausführungszeit unter einem begründeten Modell. Das beobachtete Maximum ist kein automatischer WCET-Nachweis. Für das Abschlussprojekt genügt deshalb kein grünes Gesamtetikett. Der Bericht nennt die einzelnen Ergebnisse und ihre praktischen Grenzen. Damit werden Architekturwahl, Datentransport, Zeitpfad und Fehlerreaktion zu einer nachvollziehbaren Argumentation.

Folie 50 / 50

Quellen und nächste Schritte

- Buttazzo: *Hard Real-Time Computing Systems*
- AUTOSAR: Classic-Schichten, Adaptive Platform Design
- Arm: CMSIS-RTOS2 Queue, Threadattribute
- ST: HSEM-Treiber, H755-Geräteheader

Abgabe: Papierlösungen, Integrationsbefund, Bilanz und Kriterienstatus.

Notizen 50 / 50

Giorgio C. Buttazzos Standardwerk trägt die Zeit- und Synchronisationsbegriffe des Moduls. AUTOSAR bedeutet AUTomotive Open System ARchitecture. Die verlinkten Classic- und Adaptive-Dokumente dienen zur Vertiefung des Schichtenexkurses; der Demonstrator enthält keinen AUTOSAR-Stack. CMSIS-RTOS2 ist die Echtzeitkernel-Schnittstelle des Common Microcontroller Software Interface Standard in Version zwei. Ihre Queue- und Threadverträge bestimmen Aufrufkontext, Timeout und Speichereinheiten.
STMicroelectronics dokumentiert die konkrete Hardware und die HAL, Hardware Abstraction Layer. HSEM bedeutet Hardware Semaphore. Die lokal verfügbaren öffentlichen Header und Treiber werden für die

Notizen, Fortsetzung 50 / 50

Anwendungsprüfung benutzt; ihre Version ist von der späteren Cube-Projektversion zu unterscheiden. Die externen Standardsdokumente konnten in dieser Cloud nicht erneut abgerufen werden, daher wird kein neuer revisionsspezifischer Hardware- oder Normnachweis behauptet. Modellrechnungen, Quellprüfung, Hostprüfung und Boardmessung erhalten im Ergebnisblatt unterschiedliche Bezeichnungen. Die nächsten Schritte sind Ihre eigenen Integrations- und Versuchsbefunde, nicht vorab ausgefüllte Messwerte.